

INSTITUTO TECNOLÓGICO SUPERIOR DE SAN PEDRO DE LAS COLONIAS



INGENIERÍA EN SISTEMAS COMPUTACIONALES

Grupo: 5A.

Fundamentos de Ingeniería en Software.

Análisis de Casos (CASO C01).

Docente:

Ruth Aivi Chávez Rodríguez.

“TecConect”

Integrantes del equipo:

Jonathan Alfredo Rodríguez Ramírez.

1. Presentación del caso y consecuencias

El caso C01 describe el funcionamiento operativo del sistema web de citas implementado en una clínica pequeña. Aunque la interfaz gráfica responde favorablemente al guardar consultas y carece de mensajes de error visibles, la plataforma presenta severas fallas en la gestión de concurrencia, persistencia, pruebas de software y despliegue técnico. La problemática principal radica en que el sistema permite reservar un mismo horario médico a dos recepcionistas de forma casi simultánea, carece de pistas de auditoría para rastrear cancelaciones o modificaciones, y las actualizaciones de código se publican de forma directa en producción.

Esta desarticulación técnica genera consecuencias críticas en tres niveles de la organización:

- a) Consecuencias para los usuarios (pacientes y personal de recepción): Los pacientes enfrentan retrasos en su atención médica, citas canceladas sin previo aviso y la molestia de acudir a la clínica solo para descubrir que su turno fue duplicado. Por su parte, las recepcionistas operan con estrés e incertidumbre continua, viéndose forzadas a resolver discrepancias mediante canales informales de mensajería externa.
- b) Consecuencias para el equipo de desarrollo: Los desarrolladores trabajan bajo un esquema meramente reactivo, aplicando parches directos sobre el entorno productivo sin evaluar efectos colaterales. Esto imposibilita estabilizar una versión base del sistema y multiplica el tiempo dedicado a sofocar incidentes de emergencia.
- c) Consecuencias para la organización (la clínica): Se incrementan los conflictos y discusiones entre el área médica y la recepción, se consume tiempo valioso del personal en corregir información manualmente y se deteriora la reputación y confianza institucional de la clínica.

2. Análisis de cinco problemas de ingeniería de software

Problema 1: Falta de control de concurrencia en la base de datos

- Causa: La ausencia de mecanismos de bloqueo de registros o transacciones atómicas a nivel de base de datos cuando múltiples clientes acceden a un mismo recurso.
- Consecuencia: Sobreescritura de registros y duplicidad de citas asignadas al mismo médico y horario.
- Práctica de Ingeniería de Software: Implementación de transacciones ACID y bloqueo optimista o pesimista en la capa de datos para garantizar la integridad referencial.
- Evidencia del caso: "Dos recepcionistas pueden reservar el mismo horario casi al mismo tiempo".

Problema 2: Ausencia de pistas de auditoría y trazabilidad del sistema

- Causa: Inexistencia de registros de eventos o bitácoras de auditoría internas que capturen los metadatos de las transacciones (usuario, fecha, hora y acción realizada).
- Consecuencia: Imposibilidad de determinar quién realizó una modificación, canceló un turno o introdujo un error en el sistema.
- Práctica de Ingeniería de Software: Registro de auditoría (Audit Logging) y diseño de trazabilidad de operaciones dentro del modelo de datos.
- Evidencia del caso: "Tampoco existe un registro claro de quién modificó o canceló una cita".

Problema 3: Pruebas de software deficientes y superficiales

- Causa: La validación del sistema se limita a verificar la disponibilidad visual de la pantalla sin formular planes formales de casos de prueba funcionales ni de esfuerzo.
- Consecuencia: Se liberan componentes con errores lógicos graves que solo se manifiestan cuando los usuarios reales interactúan bajo condiciones de carga cotidiana.
- Práctica de Ingeniería de Software: Definición de una estrategia formal de aseguramiento de calidad (QA) que incorpore pruebas unitarias, de integración y pruebas funcionales automatizadas (Pressman, 2010).
- Evidencia del caso: "Las pruebas se limitan a comprobar que la pantalla abra y que sea posible guardar una cita".

Problema 4: Despliegue de código directo en producción

- Causa: Falta de ambientes segregados de desarrollo y homologación, así como la carencia de un proceso estandarizado de integración continua.
- Consecuencia: Alto riesgo de interrumpir la disponibilidad total del servicio de la clínica e introducir nuevos fallos críticos durante el horario de operación.
- Práctica de Ingeniería de Software: Gestión de configuraciones, control de versiones mediante ramas y canalizaciones de despliegue continuo (CI/CD) con ambientes separados (Somerville, 2011).
- Evidencia del caso: "El equipo desarrollador publica cambios directamente en producción cuando la clínica reporta un problema urgente".

Problema 5: Inexistencia de planes de respaldo y recuperación ante desastres

- Causa: Omisión de procedimientos formales de continuidad operativa y salvaguarda periódica de la información en el ciclo de vida del software.
- Consecuencia: Vulnerabilidad total ante pérdidas definitivas de historiales médicos o incapacidad de restaurar el software tras un error crítico.
- Práctica de Ingeniería de Software: Política estructurada de copias de seguridad periódicas automatizadas y planes de recuperación ante desastres (Disaster Recovery) (Somerville, 2011).
- Evidencia del caso: "No hay un procedimiento acordado para respaldar la información, recuperar una versión anterior ni capacitar al personal".

3. Tres acciones prioritarias

Acción 1: Implementar control transaccional y validación de reservas en la base de datos

- Orden: Prioridad 1.
- Justificación: Resuelve de forma inmediata el dolor principal del negocio, evitando que sigan generándose citas empalmadas que afecten a los médicos y pacientes.
- Costo o limitación: Requiere detener temporalmente ciertas modificaciones funcionales para reestructurar la lógica de consultas y validar bloqueos en la base de datos.
- Forma de comprobar la mejora: Ejecutar pruebas de concurrencia donde dos sesiones envíen una petición simultánea para el mismo bloque horario; el sistema debe aprobar únicamente una y rechazar la segunda de manera controlada.

Acción 2: Separar el entorno de desarrollo del de producción y usar control de versiones

- Orden: Prioridad 2.
- Justificación: Detiene la introducción descontrolada de errores en el sistema real y asegura que cualquier parche urgente sea probado antes de llegar a los usuarios finales.
- Costo o limitación: Implica configurar un servidor de pruebas adicional y adaptar al equipo a flujos de trabajo más disciplinados mediante ramas de repositorio.
- Forma de comprobar la mejora: Registrar cero despliegues manuales en el entorno productivo durante un periodo de observación y documentar que cada actualización cuente con una versión previa etiquetada y respaldada.

Acción 3: Diseñar e integrar una bitácora de auditoría para modificaciones

- Orden: Prioridad 3.
- Justificación: Otorga transparencia a las operaciones, permitiendo reconstruir el historial de cambios de cada cita y eliminando disputas internas sobre autoría de cancelaciones.
- Costo o limitación: Incrementa el almacenamiento de datos del sistema y requiere modificar los formularios para ligar la sesión activa del personal a cada movimiento.
- Forma de comprobar la mejora: Revisar que ante cualquier cambio o cancelación se genere automáticamente un registro histórico que muestre usuario, stampa de tiempo y valor previo.

4. Reflexión personal

El análisis del caso C01 evidencia que el desarrollo de software no se reduce a lograr que una interfaz funcione visualmente. En proyectos anteriores tendía a concentrar la mayor parte del esfuerzo en la apariencia de las pantallas y en flujos básicos exitosos ("happy paths"), asumiendo que los usuarios interactuarían de manera aislada y ordenada. Este caso demuestra que la robustez de un sistema reside en la gestión de concurrencia, la trazabilidad y la disciplina de despliegue. Como futuro ingeniero de software, cambiaré mi enfoque estableciendo desde el inicio ambientes segregados de trabajo, diseñando pruebas antes de publicar y asegurando la integridad transaccional de los datos antes de priorizar aspectos visuales.

Referencias

Pressman, R. S. (2010). *Ingeniería del software: Un enfoque práctico*. McGraw-Hill.

Sommerville, I. (2011). *Ingeniería del Software*. Pearson Education.

• Enlace 1:

Pressman, R. S. (2010). *Ingeniería del software: Un enfoque práctico* (7.^a ed.). McGraw-Hill Interamericana Editores. <https://profesorezequielruizgarcia.wordpress.com/wp-content/uploads/2015/01/ingenieria-del-software-un-enfoque-practico-roger-s-pressman.pdf>

• Enlace 2:

Sommerville, I. (2011). *Ingeniería del software* (9.^a ed.). Pearson Educación. https://qc.scalahed.com/recursos/files/r161r/w25469w/ingdelsoftwarelibro9_compressed.pdf